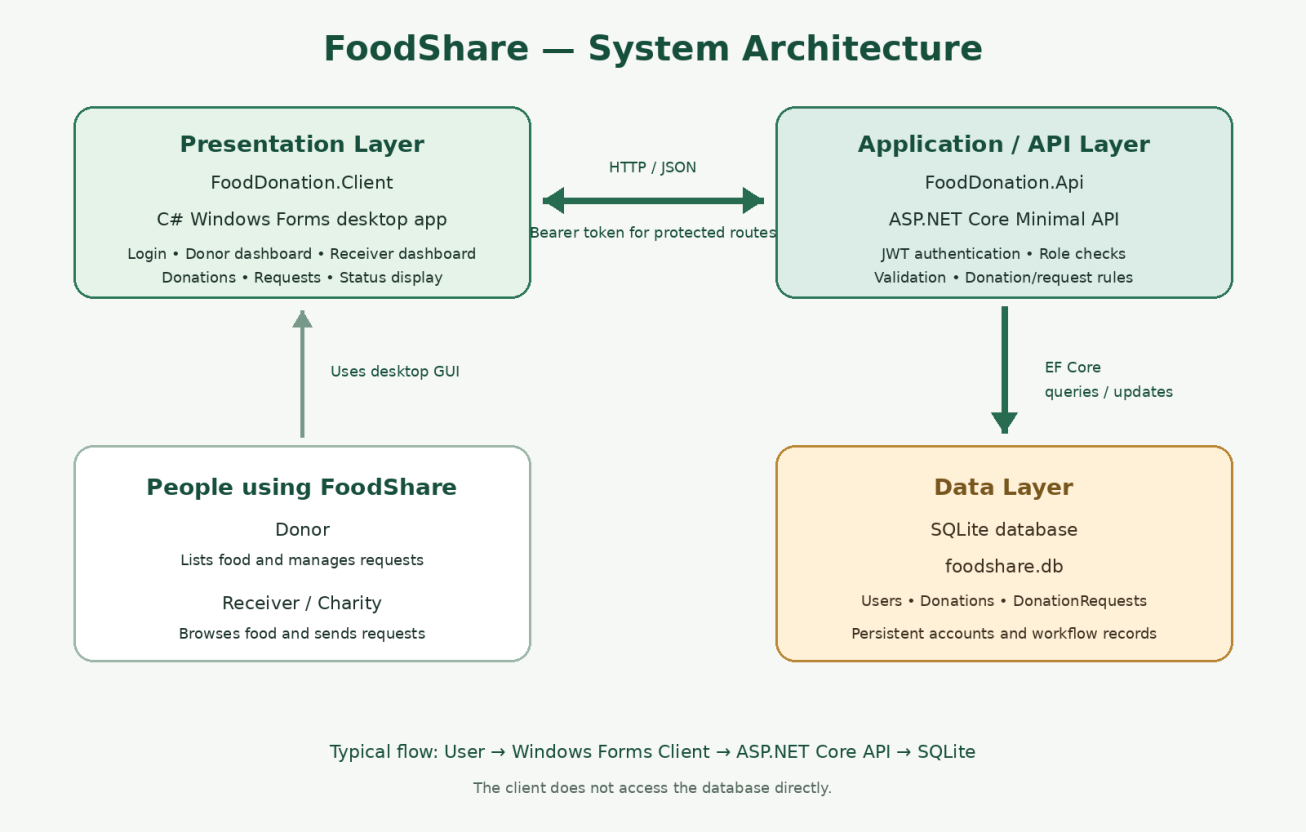
FoodShare

Food Donation Coordination System

PROJECT DOCUMENTATION



A desktop application that connects food donors with receivers and charities.

Technology: C# Windows Forms • ASP.NET Core Web API • Entity Framework Core • SQLite

Document Contents

- 1. Project Overview
- 2. Problem Statement and Proposed Solution
- 3. Objectives and Scope
- 4. Users and Functional Requirements
- 5. Technology Stack
- 6. System Architecture
- 7. Database Design
- 8. Application Workflows
- 9. API Overview
- 10. Validation and Security
- 11. Installation and Execution
- 12. Testing Plan
- 13. Limitations and Future Enhancements
- 14. Conclusion

1. Project Overview

FoodShare is a food donation coordination application designed to help people or organizations with surplus food share it with people or charities that need it. Donors can create and manage food donation listings. Receivers can browse available listings and submit requests. Donors can review incoming requests and approve or reject them. The application records accounts, donations, and requests in a local SQLite database through a separate Web API.

The solution is divided into a Windows desktop client, an ASP.NET Core API, and a SQLite data store. This separation keeps the user interface independent from the database operations and allows the API to enforce validation, authentication, and workflow rules.

2. Problem Statement and Proposed Solution

Problem statement

Usable food may be left unused while individuals and community organizations experience food insecurity. Without a coordinated process, it can be difficult to publish available food, discover it, request it, and track whether a request has been accepted.

Proposed solution

FoodShare provides a shared digital workflow: donors publish food details, receivers view available food and send requests, and donors manage requests from their own listings. Request status and donation status are stored so the participants can see the outcome.

3. Objectives and Scope

- Provide registration and login for donor and receiver accounts.
- Allow donors to create and view their own food donation listings.
- Allow donors to edit donation information where supported by the client.
- Allow receivers to browse available donations and submit a request message.
- Allow receivers to review their own requests and their statuses.
- Allow donors to view incoming requests for their donations and approve or reject pending requests.
- Persist user, donation, and request records in SQLite.
- Use role-aware API endpoints to restrict donor and receiver actions.

Scope note: FoodShare coordinates listings and requests. It does not itself provide food transport, verify food safety, process payments, or guarantee delivery. Those activities remain outside the current application.

4. Users and Functional Requirements

User / role	Main capabilities
Donor	Register/login; create and manage donation listings; view requests for owned donations; approve or reject pending requests.
Receiver / Charity	Register/login; view available food; request a donation with an optional message; view request history and statuses.
System/API	Validate input; authenticate accounts; enforce role and ownership checks; save records and update statuses.

5. Technology Stack

Technology	Purpose
C# / .NET 10 Windows Forms	Desktop user interface and donor/receiver dashboards.
ASP.NET Core Minimal API	HTTP endpoints for authentication, donations, and requests.
Entity Framework Core	Object-relational access to users, donations, and request records.
SQLite	Local persistent database stored in foodshare.db.
JWT Bearer authentication	Signed access tokens for authenticated API calls.
PasswordHasher / ASP.NET Identity	Password hashing and verification.
JSON over HTTP	Data exchange between the desktop client and API.

6. System Architecture

FoodShare uses a three-layer architecture. The presentation layer is the Windows Forms client. The application layer is the ASP.NET Core API, which performs validation and business-rule checks. The data layer is SQLite, accessed by the API using Entity Framework Core. The client communicates with the API using HTTP requests and JSON responses; it does not connect directly to SQLite.

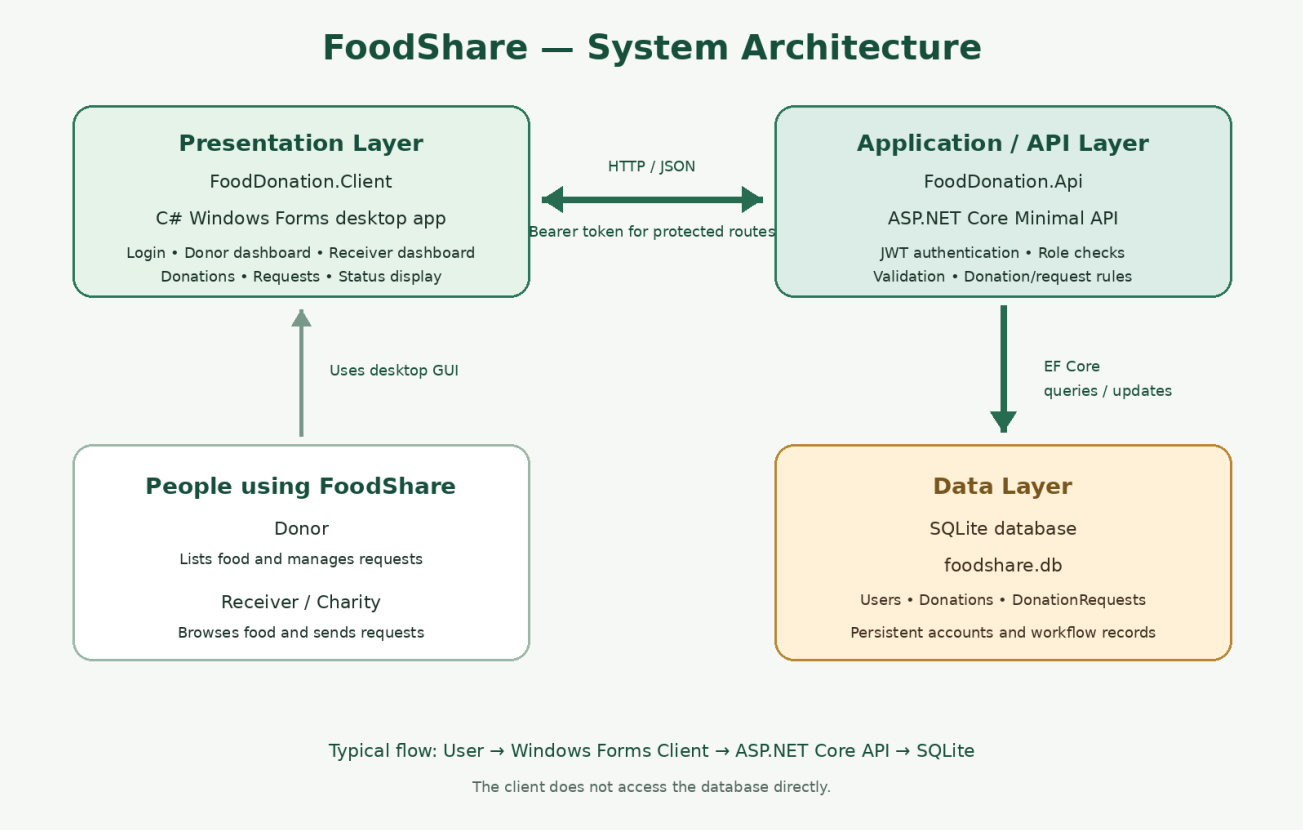


Figure 1. FoodShare logical architecture.

6.1 Main components

Component	Responsibilities
FoodDonation.Client	Displays login/register screens, dashboard panels, donation lists, request windows, status labels, and user actions.
FoodDonation.Api	Exposes endpoints, validates input, issues and validates JWTs, checks roles/ownership, applies request rules, and coordinates database operations.
FoodShareDbContext	Defines Entity Framework Core access to the application entities.
foodshare.db	Stores persistent user accounts, donation listings, and donation requests.

7. Database Design

The API initializes the database when it starts. The source includes database creation and CREATE TABLE IF NOT EXISTS statements for donation and request tables. The main entities are:

Entity	Important fields / meaning	Relationships
Users	Id, FullName, Email, NormalizedEmail, Phone, PasswordHash, Role, CreatedAtUtc	One donor can own many donations; one receiver can create many requests.
Donations	Id, DonorUserId, FoodName, Category, Quantity, PickupLocation, ExpiryDateUtc, Notes, Status, CreatedAtUtc	Each donation belongs to one donor and can have multiple requests.
DonationRequests	Id, DonationId, ReceiverUserId, Message, Status, CreatedAtUtc, UpdatedAtUtc	Each request refers to one donation and one receiver.

7.1 Relationships

- Users (donor) 1 → many Donations.
- Donations 1 → many DonationRequests over time.
- Users (receiver) 1 → many DonationRequests.
- Foreign keys connect requests to donations and receivers; donation records connect to their donor.
- A partial unique index prevents the same receiver from having multiple Pending or Approved requests for the same donation.

8. Application Workflows

8.1 Registration and login

1. The user registers with name, email, phone, role, and password.
2. The API validates the submitted information and checks whether the email already exists.
3. The password is hashed before the user record is stored.
4. During login, the API verifies the password and issues a JWT that expires after 60 minutes.
5. The client uses the token for protected API operations.

8.2 Donation creation and browsing

6. A donor signs in and creates a donation with food name, category, quantity, pickup location, expiry date, and optional notes.
7. The API associates the donation with the authenticated donor.
8. The donor can load their own donation listings.
9. A receiver can browse donations marked Available and not expired.

8.3 Request and approval lifecycle

10. A receiver selects an available donation and submits a request, optionally including a message.
11. The API verifies that the donation exists, is Available, and has not expired, then saves a Pending request.
12. The donor views incoming requests associated with the donor's own donation(s).
13. The donor can approve or reject a Pending request.
14. On approval, the request becomes Approved and the donation becomes Reserved. Other Pending requests for that same donation are changed to Rejected.
15. On rejection, the selected request becomes Rejected. The receiver can review the status in request history.

8.4 Request state diagram

Current state	Action	Result
No request	Receiver submits request	Pending
Pending	Donor approves	Approved; donation becomes Reserved
Pending	Donor rejects	Rejected
Pending (another request for same donation)	A different request is approved	Rejected automatically

9. API Overview

The following routes are defined in the API source reviewed for this documentation. Protected routes require a valid JWT Bearer token and the appropriate role.

Method	Endpoint	Purpose / access
GET	/	API status message.
GET	/api/health	Health check.
POST	/api/auth/register	Create donor or receiver account.
POST	/api/auth/login	Validate credentials and issue token.
GET	/api/donations/mine	List the authenticated donor's donations (Donor).
GET	/api/donations/available	List available donations (Receiver).
POST	/api/donations/{donationId}/requests	Create a request for a donation (Receiver).
GET	/api/donation-requests/mine	List requests made by the receiver (Receiver).
GET	/api/donations/{donationId}/requests	List requests for a donation owned by the donor (Donor).
POST	/api/donation-requests/{requestId}/approve	Approve a pending request (Donor).
POST	/api/donation-requests/{requestId}/reject	Reject a pending request (Donor).

10. Validation and Security

- Registration checks the full name, email format, phone digits, country calling code, allowed role, password strength, and duplicate email.
- Passwords are stored as hashes rather than plain text.
- JWT tokens include user identity, email, and role claims; token lifetime is 60 minutes.
- Role restrictions separate donor-only and receiver-only endpoints.
- Donor endpoints check ownership before returning or changing request records.
- Donation requests are accepted only for available, non-expired donations.
- Only Pending requests can be approved or rejected.
- The API limits request messages to 500 characters.
- The included JWT key is a development configuration value. Before real deployment, store a strong secret outside source control and configure HTTPS and suitable CORS restrictions.

11. Installation and Execution

The following steps describe a typical local development setup for the solution.

16. Install the .NET 10 SDK on the Windows development computer.
17. Extract the FoodDonationSystem project folder and open the solution in Visual Studio or a terminal.
18. Confirm the API configuration has a valid JWT key, issuer, and audience. Do not use the example development secret in production.
19. Start FoodDonation.Api. The API creates/initializes foodshare.db in its working directory when needed.
20. Start FoodDonation.Client and sign in or register as a donor or receiver.
21. For the local configuration used in development, confirm that the client API base URL matches the API's actual listening URL (the project has used http://localhost:5233).
22. To compile the solution from its root folder, run: dotnet build FoodDonationSystem.slnx

Important: The SQLite database path is relative (Data Source=foodshare.db), so the directory from which the API runs determines which database file is opened. Keep the API working directory consistent when testing persistent data.

12. Testing Plan

Test	Procedure	Expected result
Registration	Register a donor and a receiver with valid details.	Accounts are created; duplicate email is rejected.
Login	Sign in with valid and invalid credentials.	Valid credentials return a token; invalid credentials are rejected.
Donor isolation	Create donations under two donor accounts.	Each donor sees only their own donation listings.
Receiver browsing	Open the available-food view as receiver.	Available, non-expired donations are displayed.
Request creation	Receiver requests an available donation.	Request is stored with Pending status.
Incoming request	Sign in as the owning donor and open incoming requests.	The receiver's request is visible for the donor's donation.
Approval	Approve a pending request.	Request becomes Approved and donation becomes Reserved.
Competing requests	Have multiple receivers request one donation, then approve one request.	Other Pending requests for that donation become Rejected.
Rejection	Reject a pending request.	Request status becomes Rejected.
Expiry validation	Attempt to request an expired donation.	API refuses the request.
Persistence	Close and restart the application/API.	Previously saved records remain in the same SQLite database.

13. Limitations and Future Enhancements

The current project focuses on coordinating donation listings and requests. Possible future enhancements include:

- Configurable API URL and deployment settings for using the client from another computer.
- Automated tests for API validation, permissions, and request state transitions.
- Notifications or a visible pending-request count on the donor dashboard.
- A completion/collection confirmation step after approval.
- Improved food safety fields such as storage conditions, allergen information, and preparation time.
- Audit history and administrative reporting.
- Production deployment with HTTPS, a securely managed JWT secret, and restrictive CORS settings.

14. Conclusion

FoodShare demonstrates a practical client–API–database architecture for coordinating food donations. It supports separate donor and receiver roles, persistent donation listings, request tracking, and donor decisions. The separation between the Windows Forms client, ASP.NET Core API, and SQLite database makes the application easier to understand, maintain, and extend. With further deployment, testing, and food-safety features, it could be developed into a more complete community food-sharing solution.

Appendix A — Suggested Demonstration Sequence

23. Start the API and then open the desktop client.
24. Sign in as a donor and create a sample donation with a future expiry date.
25. Sign out and sign in as a receiver; locate the donation and submit a request.
26. Sign out and sign in as the donor; open incoming requests and approve or reject the receiver's request.
27. Return to the receiver account and show the updated request status.
28. Explain that the data is persisted in SQLite and requests are processed through the API.